

March 2026 LILUG Meeting

March 10th, 2026 @ [Digital Ballpark](#) Pace-notes by [Chris Trimble](#)

News & Small Talk

- Are you an Adult? Age Verification laws are being considered around the world.

Main Discussion: Matthew Newhall Presents Ansible

Structural Concepts

In the beginning, there were programming languages and shell scripts. Then, SSH, telnet, and kickstart with PXE happened. [BOFH](#) and shell scripts were common place.

"High level languages made bad things happen fast".

Then, automation systems like Puppet happened which allowed some ability to run safely.

Ansible allows failsafe, repeatable automation to run on multiple platforms without installing anything on the remote devices.

Tech Powering Ansible

- YAML
- JSON
- SSH
- Jinja2
- Python
- Ansible Galaxy
- Ansible vault
- Local shell commands (such as Bash)
- Ansible specific YAML structures

Fundamental/Prerequisite Skills to Learn

- SSH
- Git practices, norms, & limitations

Other Helpful Tools for Ansible

- Ara: Ara Record Ansible
 - Makes records for when playbooks run and how/who completes actions
- Git: Version Control
- Pip: Python package installer

Purposes of Ansible

Normalized automation:

- normalized data
- normalized actions
 - **Perpetually automate to keep the working environment honest.**
- *Idempotency*: run the script over and over without negative effects. If the action is already done it will not do it again unless something is changed.
- Infrastructure as Code (IaC)
- Continuous Integration (CI)
- Continuous Development (CD)

Transferring Data

Transfers can either be done with direct commands in the shell or "become", which allows Ansible to run the command as a certain user.

- File operations
- SSH
- APIs
- Database calls

Example file structure:

We can make file structure as simple or complex as needed, but the primary thing is that all code runs inside a parent folder, everything is then referenced relatively using definitions in playbooks as well as `ansible.cfg`

~/LILUG/Ansible vs ~/LILUG/git/Ansible

- There may be different paths, where Ansible runs will determine which code it can pull from.

Common directories inside a parent directory include:

- hosts
- playbooks
- roles
- .ansible (config for the parent folder)
- defaults
- vars
- inventory

To include different files in a Playbook, an *include* function can call for them using relative paths.

Running Ansible Manually

1. Change into the directory holding Ansible code:
 - `cd ~/Ansible`
2. If needed, sync the newest playbooks or code you wrote using Git or your preferred versioning:
 - `git pull`
3. Run the playbook:
 - `ansible-playbook /playbooks/LILUG-things.yml`

Likely Ansible Flags

```
-l interestinghostname interestinghostname2 --ask-vault-password -u root --extra-
vars LILUG=tonight -b specialuser --ask-become-pass -C # Do not commit just
report/estimate what will happen -D #Diff files that may change -vvv # make
terminal more verbose
```

Roles

- task/main.yml - a list of tasks that the role provides to the play for execution.
- handlers/main.yml - handlers that are imported into the parent play for use by the role or other roles and tasks in the play.
- vars/main.yml - high precedence variables provided by the role to the play.
- defaults/main.yml - very low precedence values provided by the roles. A role's own defaults will take priority over other role defaults, but any/all other variable sources.
- files/stuff.txt - one or more files that are available for the role and its children.
- nnnn templates/something.j2 - template to use in the role or child roles.
- meta/main.yml - metadata for the role, including role dependencies and the optional Galaxy metadata such as platforms supported. This is required for uploading into galaxy as a standalone role, but not for using the role in your playbook.

Storing Passwords

Running commands in Ansible will often need passwords, and it is not a good idea to store these in plain text. Using Ansible Vault is for managing passwords.

Ansible Galaxy

Galaxy is a tool for managing modules in Ansible.

- List Modules: `ansible-galaxy collections list`
 - You can also do something like `ls ~/.ansible/collections/ansible_collections`
- Update a Module: `ansible-galaxy collection install community.docker --upgrade`
- Install a module (forcefully): `ansible-galaxy collection install community.docker --force`

Programming Concepts

When multiple data sources, APIs, or complex setups, you will need programming outside of basic YAML for Playbooks.

Things needed for Playbooks that respect data privacy:

- High level programming languages
- Shell languages
- Data structures
 - *Includes* are pre-processed
 - *Imports* are at runtime

An Example Playbook

```
bare-bones.yml --- - name: The Name of the Playbook hosts: localhost tasks: - name:
some task ansible.builtin.file: path: /tmp/foo.conf state: touch mode: '600'
```

Third party tools to add onto Ansible

- git-leaks: scans for secrets in version control
- BFG-repo-cleaner: removes passwords and secrets
- Terraform: for provisioning (in conjunction with Ansible)
- Netbox: inventory management
- Molecule: staging, development, and testing
- Gitlab/Github/git: version control
- Rundeck: cron server with tiered access controls
- jq: sed for JSON
- Ansible-lint: indicates best practices for Ansible

Additional Reading

[Ansible for DevOps, by Jeff Geerling](#)